



# Bone Marrow Transplant Outcome Analysis & Survivor Modeling

Group 3

Coauthor 1, Justin Maroldi, Coauthor 2

# The Dataset



UC Irvine  
Machine Learning  
Repository



What is it: Publicly available dataset from

UC Irvine that contains medical and survival data for children that have undergone bone marrow transplants.

Purpose: To support researchers in the medical field that are building models around survival analysis and general treatment effectiveness.

Importance: Due to the vast effects surgery can have on people, have a moderately large number of observations, containing key biomarkers, that have been professionally tracked and verified, allows for realistic model building, in comparison to simulated data.

# Preliminary Exploration and Conclusions

Presences of missing values: In the dataset there are several columns that have missing values that should be accounted for before modeling

Small dataset size: The dataset only contains 187 entries which could lead to inadequate data for modeling, especially after splitting it into smaller sets

Overall statistics: The data does not appear to be too normal and there are imbalances in the data

```
> summary(data)
Recipientgender StenCellsource Donorage Donorage35 IIIV Gendermatch DonorABO RecipientABO RecipientRh
0: 75          0: 42          Min.   :18.65  0:104      0: 75      0:155      -1:28      -1: 50      0: 27
1:112          1:145          1st Qu.:27.04  1: 83      1:112      1: 32      0: 73      0: 48      1: 158
                                   Median :33.55                                   1: 71      1: 75      NA's: 2
                                   Mean   :33.47                                   2: 15      2: 13      NA's: 1
                                   3rd Qu.:40.12                                   NA's: 1
                                   Max.   :55.55

ABOMatch CMVstatus DonorCMV RecipientCMV Disease Riskgroup Txpostrelapse Diseasegroup HLAmatch
0: 52      0: 48      0: 113      0: 73      ALL      :68      0:118      0:164      0: 32      0:94
1: 134      1: 27      1: 72      1: 100      AML      :33      1: 69      1: 23      1:155      1:65
NA's: 1      2: 57      NA's: 2      NA's: 14      chronic :45      NA's: 1
                                   lymphoma :9
                                   nonmalignant:32
                                   NA's:16

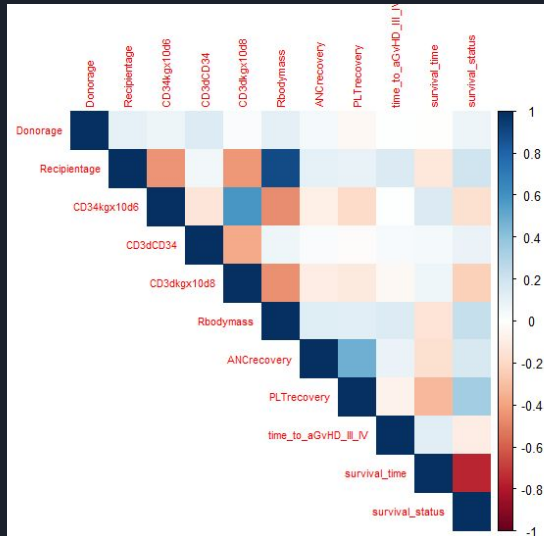
HLAmismatch Antigen Alel HLAgrI Recipientage Recipientage10 Recipientageint Relapse agVHDIIIIV extcGVHD
0:159      -1: 93      -1: 93      0:94      Min.   : 0.600  0:99      0:47      0:159      0: 40      0: 28
1: 28      0: 21      0: 54      1:42      1st Qu.: 5.050  1:88      1:51      1: 28      1:147      1: 128
                                   2: 14      Median : 9.600
                                   3: 9       Mean   : 9.932
                                   4:19      3rd Qu.:14.050
                                   5: 4       Max.   :20.200
                                   7: 5
                                   NA's: 1
                                   NA's: 1
                                   NA's: 5
                                   NA's: 5
                                   NA's: 2

CD34kgx10d6 CD3dCD34 CD3dkgx10d8 Rbodymass ANCrecovery PLTcrecovery
Min. : 0.79 Min. : 0.2041 Min. : 0.040 Min. : 6.0 Min. : 9 Min. : 9
1st Qu.: 5.35 1st Qu.: 1.7867 1st Qu.: 1.688 1st Qu.: 19.0 1st Qu.: 13 1st Qu.: 16
Median : 9.72 Median : 2.7345 Median : 4.325 Median : 33.0 Median : 15 Median : 21
Mean :11.89 Mean : 5.3851 Mean : 4.746 Mean : 35.8 Mean : 26753 Mean : 90938
3rd Qu.:15.41 3rd Qu.: 5.8236 3rd Qu.: 6.785 3rd Qu.: 50.6 3rd Qu.: 17 3rd Qu.: 37
Max. :57.78 Max. :99.5610 Max. :20.020 Max. :103.4 Max. :1000000 Max. :1000000
NA's :5 NA's :5 NA's :2

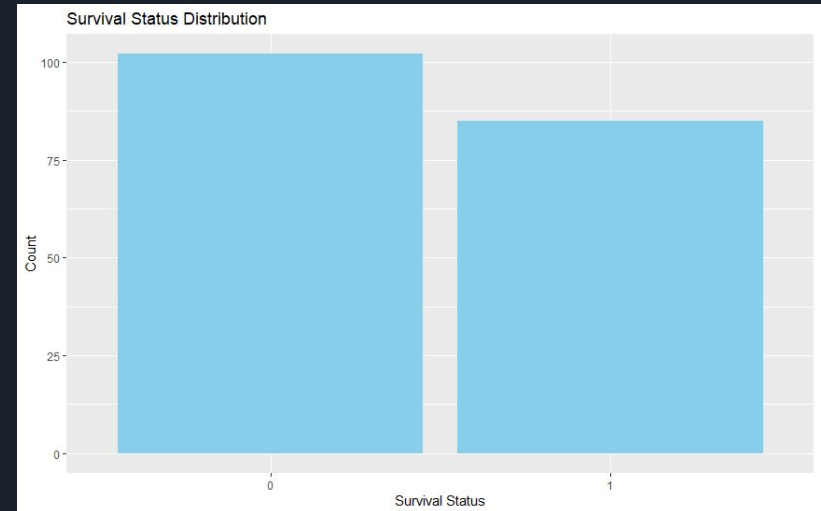
time_to_agVHD_III_IV survival_time survival_status
Min. : 10 Min. : 6.0 Min. :0.0000
1st Qu.:1000000 1st Qu.: 168.5 1st Qu.:0.0000
Median :1000000 Median : 676.0 Median :0.0000
Mean : 775408 Mean : 938.7 Mean :0.4545
3rd Qu.:1000000 3rd Qu.:1604.0 3rd Qu.:1.0000
Max. :1000000 Max. :3364.0 Max. :1.0000
```

# Preliminary Exploration and Conclusions Contd.

Correlation: There is significant correlation between several important variables and could impair models



Class imbalance: There is a minor imbalance between classes which could cause prediction problems





# Other Considerations

Variables: The variables that are present in the dataset cannot be used outright.

There are variables that are only available after the surgery (ex. Survival\_Time). To prevent data leakage we initially limited our dataset, removing all variables that wouldn't be present by the time post-surgery.\*

Data size: Another consideration that we had to keep in mind was the size of our dataset, we only had 187 observations in our dataset so the models that are being trained (given a 70/30 train/test split) don't have enough data to be as robust as possible.

Class balance: The binary class variables are also not completely 50/50, which in a small sample size further limits predictive robustness. (ex. The predictor 'survival\_status' variable has a split of 102 - 0's and 85 - 1's which is further reduced once we get to the data cleaning stage of our models)

\* ("Recipientgender", "Stemcellsource", "Donorage", "Donorage35", "Gendermatch", "DonorABO", "RecipientABO", "RecipientRh", "ABOMatch", "CMVstatus", "DonorCMV", "RecipientCMV", "Disease", "Riskgroup", "Diseasegroup", "HLAmatch", "HLAmismatch", "Antigen", "Allele", "HLAgr1", "Recipientage", "Recipientage10", "Recipientageint", "CD34kgx10d6", "CD3dCD34", "CD3dkgx10d8", "Rbodymass", 'survival\_status' )

# Model A: Random Forest (Variable Selection)

Random Forest Models weigh the importance of variables on its own through the tree building process, so I don't need to do any variable selection explicitly. However there is a need to deal with the missing values in the dataset.

## Model A (Removing the columns that have missing values)

```
In [15]: # Making a dataframe without the missing columns
model_A = filtered_df_1.dropna(axis=1)
print(model_A.shape)
print("Removed Columns", set(filtered_df.columns) - set(model_A.columns))
model_A

(187, 17)
Removed Columns {'DonorCMV', 'Allele', 'CD3dCD34', 'RecipientCMV', 'RecipientRh', 'CMVstatus', 'Rbodymass', 'CD3dkgx10d8', 'RecipientABO', 'Antigen', 'ABOmatch'}
```

## Model B (Removing the rows with missing value)

```
In [17]: # Making a dataframe without the rows that are missing values
model_B = filtered_df_1.dropna()
print(model_B.shape)
model_B

(164, 28)
```

# Model A: Random Forest Model

In order to optimize the performance of both models I utilized grid search, a function that tests out all parameters in a parameter grid to see which RF parameters result in the highest accuracy rate.

```
In [9]: # Setting up parameters for optimizing the random forest model
        from sklearn.model_selection import GridSearchCV
        param_grid = {
            'n_estimators': [50, 100, 200, 300, 500],
            'max_depth': [5, 10, 20, 30, 50, None],
            'min_samples_split': [2, 5, 10, 20],
            'min_samples_leaf': [1, 2, 5, 10, 20],
            'max_features': ['auto', 'sqrt', 'log2', None]
        }
```

Model A

|                             |           |        |          |         |  |
|-----------------------------|-----------|--------|----------|---------|--|
| MODEL A                     |           |        |          |         |  |
| Accuracy: 0.543859649122807 |           |        |          |         |  |
| Classification Report:      |           |        |          |         |  |
|                             | precision | recall | f1-score | support |  |
| 0                           | 0.59      | 0.59   | 0.59     | 32      |  |
| 1                           | 0.48      | 0.48   | 0.48     | 25      |  |
| accuracy                    |           |        | 0.54     | 57      |  |
| macro avg                   | 0.54      | 0.54   | 0.54     | 57      |  |
| weighted avg                | 0.54      | 0.54   | 0.54     | 57      |  |

Model B

|                        |           |        |          |         |  |
|------------------------|-----------|--------|----------|---------|--|
| MODEL B                |           |        |          |         |  |
| Accuracy: 0.48         |           |        |          |         |  |
| Classification Report: |           |        |          |         |  |
|                        | precision | recall | f1-score | support |  |
| 0                      | 0.47      | 0.75   | 0.58     | 24      |  |
| 1                      | 0.50      | 0.23   | 0.32     | 26      |  |
| accuracy               |           |        | 0.48     | 50      |  |
| macro avg              | 0.49      | 0.49   | 0.45     | 50      |  |
| weighted avg           | 0.49      | 0.48   | 0.44     | 50      |  |

## Model B: KNN (Variable Selection)

Selected variables:

Recipientgender, Stemcellsource, Donorage35, Gendermatch, DonorABO, RecipientABO, RecipientRh, ABOmatch, DonorCMV, RecipientCMV, Disease, Riskgroup, HLA mismatch, Antigen, Allele, Recipientage10, CD34kgx10d6, CD3dCD34", CD3dkgx10d8, survival\_status

These variables were selected to minimize the correlation between variables. This will help the KNN model by reducing bias and improving modeling quality

```
# Remove correlated variables
cor_matrix <- cor(data[, -which(names(data) == "survival_status")])
highly_correlated <- findCorrelation(cor_matrix, cutoff = 0.75)
data_reduced <- data[, -highly_correlated]
data <- data_reduced
```

|                |                          |
|----------------|--------------------------|
| ▶ data         | 187 obs. of 28 variables |
| ▶ data_reduced | 187 obs. of 20 variables |



## Model B: KNN

Before modeling NAs are dealt with by finding the median for continuous columns and mode for discrete columns. Then the columns present before a transplant is conducted are selected to be the starting point for variable selection. Final set of variables are found by calculating the correlation between variables and remove the heavily correlated ones. The data can then be split into a training, validating, and testing set with separation of predictor and response columns. Model can then be trained on the training set. Using the model the validation set is used to find the best threshold to classify predictions using F1 score and ROC as metrics to identify the best threshold. Using the threshold found the test set is used to access model performance.

### Confusion Matrix and Statistics

|   | 0  | 1  |
|---|----|----|
| 0 | 28 | 20 |
| 1 | 3  | 6  |

Accuracy : 0.5965

95% CI : (0.4582, 0.7244)

No Information Rate : 0.5439

P-Value [Acc > NIR] : 0.2539664

Kappa : 0.1415

McNemar's Test P-value : 0.0008492

Sensitivity : 0.9032

Specificity : 0.2308

Pos Pred Value : 0.5833

Neg Pred Value : 0.6667

Prevalence : 0.5439

Detection Rate : 0.4912

Detection Prevalence : 0.8421

Balanced Accuracy : 0.5670

'Positive' Class : 0

# Model C: Logistic Regression (Variable Selection)

- Initial Selection: Stepwise regression to identify candidate predictors.
- Refinement Process:
  - Multicollinearity Check: Removed variables with high collinearity (using VIF analysis).
  - More iterative adjustments to retain interpretability and predictive performance.
- Outcome: Best-fit model with significant predictors, balancing accuracy and simplicity.
- Final selected variables are RecipientRh, RecipientCMV, Riskgroup, Recipientage and DonorCMV.

```
> vif(final_model)
```

| Stemcellsource | DonorCMV     | RecipientCMV | Riskgroup            | Recipientage  |
|----------------|--------------|--------------|----------------------|---------------|
| 4.035091e+02   | 2.902223e+02 | 5.544079e+02 | 1.125640e+01         | 1.134534e+01  |
| Relapse        | aGvHDIIIIV   | extcGvHD     | time_to_aGvHD_III_IV | survival_time |
| 6.118045e+02   | 3.883254e+11 | 4.364866e+00 | 3.883257e+11         | 6.663285e+01  |

```
> vif(reduced_model)
```

| RecipientRh | RecipientCMV | Riskgroup | Recipientage | DonorCMV |
|-------------|--------------|-----------|--------------|----------|
| 1.008200    | 1.007409     | 1.009368  | 1.006028     | 1.007162 |



# Model C: Logistic Regression Workflow and Improvements

1. Data Preprocessing:
  - a. Handled missing values using k-NN imputation.
  - b. Split the dataset into 70/30 train/test sets for model evaluation.
2. Initial Model Development:
  - a. Performed stepwise regression to select key predictors.
  - b. Built an initial logistic regression model
3. Model Refinement:
  - a. Addressed multicollinearity (e.g., removed variables with high VIF).
  - b. Improved model interpretability and predictive accuracy.
4. Evaluation and Results:
  - a. Better fit: Improved QQ plot from original model indicates enhanced residual normality.



# Which Is Better?

## 1. Model A: Random Forest

- a. Handles non-linear relationships well and ensemble method reduces overfitting.
- b. Struggles with small datasets like ours.
- c. Confusion matrix accuracy is  $\sim 0.543$ .

## 2. Model B: KNN

- a. Selected 20 variables to minimize correlation.
- b. Preprocessed with median/mode imputation.
- c. Optimized using F1 score and ROC
- d. Confusion matrix accuracy is  $\sim 0.596$ .

## 3. Model C: Logistic Regression

- a. Variables selected via stepwise regression and VIF analysis.
- b. Preprocessed with k-NN imputation (70/30 split).
- c. Best-fit model with RecipientRh, RecipientCMV, Riskgroup, Recipientage, DonorCMV.
- d. Confusion matrix accuracy is  $\sim 0.59$



# Results From Predictive Analysis

- Best Model: KNN (Model B)
  - Achieved the highest accuracy (0.596) on the data.
- Key Takeaways:
  - Model A (Random Forest) struggled due to the small dataset, leading to overfitting and low test accuracy.
  - Model B (KNN) performed moderately but potentially was limited by sensitivity to variable scaling and correlation.
  - Model C (Logistic Regression) had a refined variable selection method. Struggled due to insufficient data and potentially perfect separation.
- Conclusion:
  - KNN (Model B) is the most reliable model for predicting in the UCI Bone Marrow dataset.



# Future Direction

Even though the best model found has been proven to be accurate it can be still be improved. There are several avenues to pursue:

Collect Additional Data:

The dataset only contains 187 entries which doesn't provide much data to work with for the model to learn

Try New Models:

Using more advanced models could lead to a better prediction outcomes with the limited data